



Instituto Tecnológico de Buenos Aires

72.20 - Redes de Información

Grupo: 5

Integrantes:

Gonzalez Cornet, Josefina

Legajo: 64550

jgonzalezcornet@itba.edu.ar

Hillar, Conrado

Legajo: 64633

chillar@itba.edu.ar

Alonso, Agustin

Legajo: 63316

agusalonso@itba.edu.ar

Fecha de entrega: 12 de mayo de 2026

1C - 2026

1. Problemática y contexto

La arquitectura de “The Store” no contempla un mecanismo centralizado de inspección y filtrado del tráfico HTTP en el borde, por lo que los endpoints expuestos pueden ser explotados mediante ataques de Capa 7. Esto permite que solicitudes maliciosas alcancen directamente los servicios backend, comprometiendo la disponibilidad, integridad y rendimiento del sistema.

2. Diseño de la solución

Se propone incorporar un Web Application Firewall (WAF) como componente perimetral de la arquitectura, actuando como único punto de entrada para el tráfico HTTP/HTTPS externo.

La solución seleccionada consiste en utilizar Nginx como reverse proxy junto con ModSecurity como motor de análisis, desplegándose sobre el nginx-ingress controller existente en el clúster de Kubernetes.

El flujo de tráfico es el siguiente: las solicitudes ingresan inicialmente al WAF, donde son analizadas en tiempo real. Aquellas consideradas legítimas según las reglas configuradas son reenviadas hacia los servicios internos, mientras que las maliciosas son bloqueadas antes de alcanzar los microservicios.

Adicionalmente, el WAF genera logs estructurados de las transacciones analizadas, incluyendo información como IP de origen, endpoint accedido, payload inspeccionado y regla aplicada, permitiendo auditar y analizar los eventos detectados.

3. Scope del POC y Casos de Uso

El alcance del Proof of Concept (POC) consiste en demostrar el correcto funcionamiento del WAF en un entorno controlado, validando su capacidad para detectar y bloquear tráfico malicioso sin afectar el comportamiento normal de la aplicación.

A continuación se detallan distintas vulnerabilidades presentes en la implementación actual de “The Store”, junto con sus respectivas reglas de mitigación.

3.1. Vulnerabilidad en Control de Accesos (IDOR)

El control de acceso es responsable de garantizar que los usuarios solo puedan acceder a los recursos que les corresponden. Una falla en este mecanismo permite que un atacante acceda a información o funcionalidades de otros usuarios simplemente manipulando identificadores (Insecure Direct Object Reference).

Casos identificados

3.1.1 El endpoint `/proxy/carts/{customerId}` no valida que el usuario autenticado sea propietario del carrito consultado, permitiendo acceder a información de otros usuarios mediante la manipulación del parámetro `customerId`.

3.1.2 El endpoint `/proxy/orders/{customerId}` presenta el mismo comportamiento, permitiendo consultar órdenes asociadas a cualquier usuario.

Mitigación con WAF

Debido a que la vulnerabilidad corresponde a un problema de autorización implementado a nivel aplicación, el WAF actuará como mecanismo de virtual patching, restringiendo el acceso externo a los endpoints vulnerables y reduciendo la superficie de exposición.

```
SecRule REQUEST_URI "@beginsWith /proxy/"
```

```
"id:1001,phase:1,deny,status:403,msg:'Access to internal proxy endpoints  
blocked'"
```

Caso de uso

Se ejecutarán requests sobre los endpoints vulnerables tanto en un entorno con WAF como sin WAF para evidenciar el bloqueo del acceso externo: `curl http://{ip}/proxy/carts/{customerId}` y `curl http://{ip}/proxy/orders/{customerId}`.

3.2. Exposición de Endpoints Administrativos

El diseño inseguro implica la exposición de funcionalidades críticas sin controles adecuados. En este caso, endpoints de debugging o administración están accesibles públicamente, permitiendo interrumpir el servicio, manipular el sistema y provocar comportamientos no previstos.

Casos identificados

3.2.1 Los endpoints `/utility/panic` y `/utility/health/down` permiten afectar directamente la disponibilidad del sistema (DoS). El primero provoca el crash de la aplicación, mientras que el segundo marca el pod de UI como unhealthy, generando respuestas 502 hasta que Kubernetes lo reinicie.

3.2.2 El endpoint `/utility/store` permite almacenar datos arbitrarios controlados por el usuario, posibilitando persistencia de información no autorizada o llenado de disco.

3.2.3 El endpoint `/utility/stress/{iterations}` ejecuta un bucle de cómputo cuya cantidad de iteraciones es definida por el usuario, permitiendo ataques de consumo excesivo de CPU (CPU-burn) que degradan el rendimiento de la aplicación y pueden provocar timeouts o caídas bajo carga.

Mitigación con WAF

Se restringirá el acceso a los endpoints administrativos mediante reglas custom que permitan únicamente requests provenientes de IPs autorizadas.

```
SecRule REQUEST_URI "@beginsWith /utility/" "id:2002,phase:1,deny,status:403,chain"
```

```
SecRule REQUEST_HEADERS:X-Forwarded-For "!@ipMatch {...}"
```

De esta manera, cualquier request proveniente de IPs no privilegiadas será bloqueada antes de alcanzar la aplicación. Para evitar spoofing del header X-Forwarded-For, nginx sobrescribirá dicho valor utilizando la IP externa real del cliente.

Caso de uso

Para demostrar el impacto de los endpoints administrativos expuestos, se ejecutarán las siguientes requests: `curl -i http://{ip}/utility/panic`, `curl -i http://{ip}/utility/health/down` y `curl -i http://{ip}/utility/stress/2000000`.

Se evidenciará como, en ausencia del WAF, estos endpoints permiten que se lleve a cabo el ataque. Con el WAF habilitado, las requests serán bloqueadas antes de alcanzar la aplicación. Adicionalmente, se demostrará la escritura arbitraria de datos mediante `curl -X POST -H "Content-Type: application/json" -d '{"a":"b"}' http://{ip}/utility/store`. La request devolverá un identificador que luego podrá utilizarse para recuperar la información almacenada mediante `curl http://{ip}/utility/store/{hash}`.

3.3. Exposición de información sensible

La exposición de información interna del sistema puede facilitar ataques posteriores, permitiendo al atacante comprender la arquitectura, servicios y dependencias y realizar ataques dirigidos.

Casos identificados

3.3.1 El endpoint `/info` expone metadata interna del sistema.

3.3.2 El endpoint `/topology` permite obtener la estructura de microservicios de la aplicación.

3.3.3 El endpoint `/utility/headers` expone headers internos del proxy y valores sensibles asociados a la sesión, incluyendo X-Session-ID, X-Forwarded-*, X-Real-IP y X-Request-ID.

Mitigación con WAF

Se bloqueará el acceso externo a endpoints sensibles mediante reglas custom específicas:

```
SecRule REQUEST_URI "@streq /info" "id:1003,phase:1,deny,status:403,msg:'Info endpoint blocked'"
```

```
SecRule REQUEST_URI "@streq /topology" "id:1004,phase:1,deny,status:403,msg:'Topology endpoint blocked'"
```

El caso 3.3.3 queda cubierto por las reglas aplicadas sobre el grupo de endpoints administrativos.

Caso de uso

Para evidenciar la vulnerabilidad y su mitigación, se ejecutarán requests contra los endpoints expuestos tanto en un entorno con WAF como sin WAF: `curl -s http://{ip}/topology`, `curl -s http://{ip}/info` y `curl http://{ip}/utility/headers`. En ausencia del WAF, las requests permiten obtener información sensible sobre la arquitectura y configuración interna del sistema. Con el WAF habilitado, las requests serán bloqueadas antes de alcanzar la aplicación.

3.4. Protección de inputs

Si bien en el análisis realizado no se identificaron vulnerabilidades activas de tipo SQL Injection, XSS o Path Traversal, se decidió extender el servicio de Catálogo incorporando una barra de búsqueda y un endpoint adicional de acceso a archivos sin mecanismos de sanitización ni validación de inputs. El objetivo es demostrar capacidades de inspección Layer 7 del WAF sobre query parameters, payloads HTTP y headers.

Casos identificados

3.4.1 Con el endpoint `GET /catalog/search?q=` se hace una búsqueda según el query del usuario sin realizar sanitización, permitiendo realizar SQL Injection y XSS.

3.4.2 Con el endpoint `GET /catalog/image?file=` se busca y retorna el archivo especificado sin realizar ninguna verificación sobre la validez de la request, permitiendo realizar Path Traversal.

3.4.3 Detección de herramientas automatizadas: los atacantes suelen utilizar herramientas automáticas de scanning y explotación como sqlmap o Nikto, las cuales incluyen firmas en headers HTTP como User-Agent.

Mitigación con WAF

Para mitigar este tipo de vulnerabilidades y otros ataques web comunes, se utilizará el OWASP Core Rule Set (CRS) como base de reglas de seguridad sobre ModSecurity, inicialmente configurado con paranoia level 2, (según la documentación del CRS¹, el mismo es adecuado para un “*off-the-shelf online shop*”). Se analizará la aparición de falsos positivos, y se establecerán *rule exclusions* (reglas que habilitan ciertos requests no maliciosos que se suelen bloquear) acordes para reducirlos.

Adicionalmente, se realizará un análisis del mecanismo de *anomaly scoring*, variando el parámetro *inbound_anomaly_score_threshold* (por default seteado en 5, aunque WAFs comerciales como Cloudflare usan valores mucho más altos) en caso de que se presenten demasiados falsos positivos. Este proceso se llama *false-positive tuning*².

Por último, se incorporará una regla custom para bloquear herramientas automatizadas identificadas mediante el header User-Agent:

```
SecRule REQUEST_HEADERS:User-Agent "@rx (?i:(sqlmap|nikto|nmap|wpscan))" \
"id:4001,phase:1,deny,status:403,msg:'Malicious scanner detected'"
```

Caso de uso

¹ https://coreruleset.org/docs/2-how-crs-works/2-2-paranoia_levels/

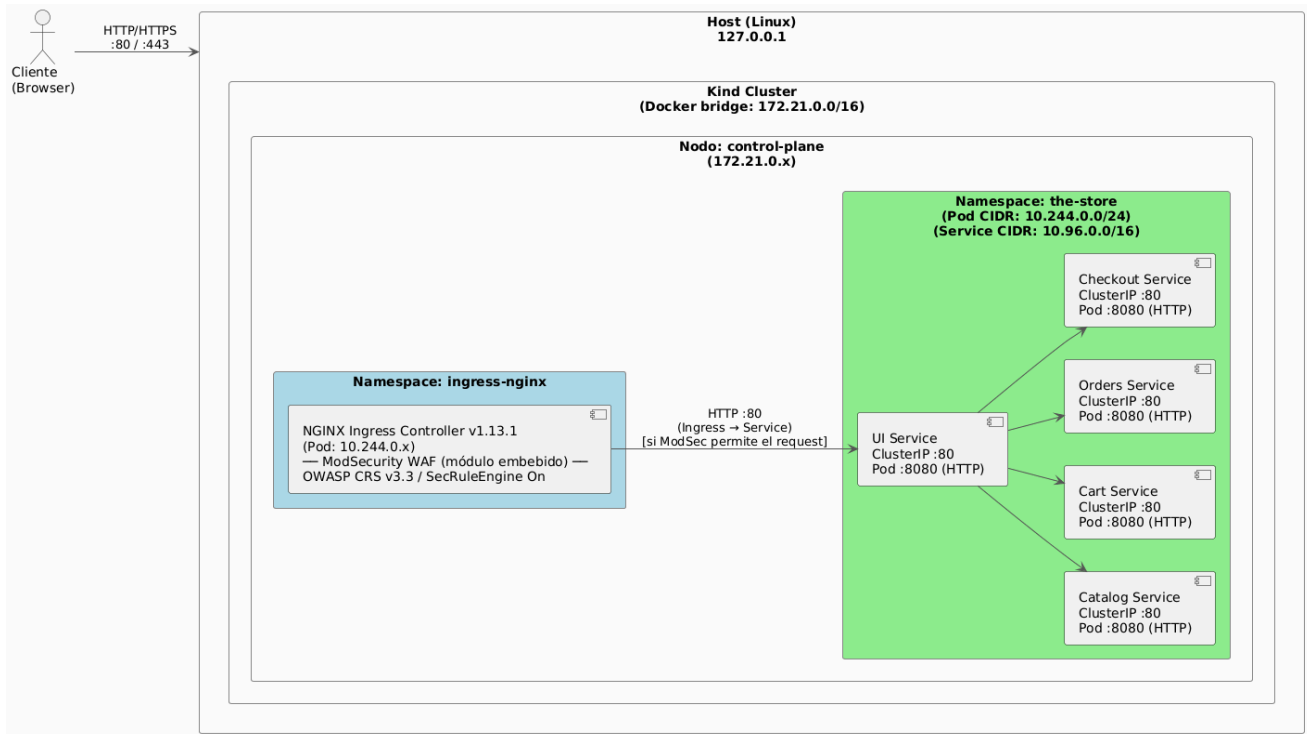
² https://coreruleset.org/docs/2-how-crs-works/2-1-anomaly_scoring/

Se ejecutarán payloads representativos de SQL Injection, XSS, Path Traversal y detección de scanners automatizados, por ejemplo:

- SQLi: `curl "http://{ip}/catalog/search?q=' OR 1=1--"` y `curl "http://{ip}/catalog/search?q=' UNION SELECT password FROM users--"`
- XSS: `curl "http://{ip}/catalog/search?q=<img src=x onerror=alert(1)>"`
- Traversal: `curl "http://{ip}/catalog/image?file=../../../../etc/passwd"` o `curl "http://{ip}/catalog/image?file=../../../../proc/self/environ"`
- Scanner detection: `curl "http://{ip}/catalog/search?q=test" -H "User-Agent: sqlmap"`

En todos los casos se comparará el comportamiento entre un entorno con WAF y otro sin WAF, analizando además los logs generados por ModSecurity y los anomaly scores asociados.

4. Diagrama de arquitectura



5. Alternativas consideradas

Se analizaron OpenWAF, Shadow Daemon y ModSecurity, evaluando su adecuación al contexto de la aplicación y al alcance del trabajo.

OpenWAF³ fue descartado debido a su bajo nivel de mantenimiento y actualización, además de no contar con integración estandarizada con OWASP Core Rule Set (CRS), lo que implica mayor complejidad de configuración y menor previsibilidad frente a amenazas conocidas.

En cuanto a Shadow Daemon⁴, su integración está orientada principalmente a lenguajes como PHP, Python y Perl. Su utilización en un entorno basado en Java y Node.js como “The Store” requeriría desarrollar conectores ad hoc, incrementando significativamente la complejidad de la solución.

Finalmente, ModSecurity fue seleccionado debido a su madurez, mantenimiento activo e integración nativa con OWASP CRS, permitiendo una incorporación más simple y estandarizada dentro de la arquitectura propuesta.

³ <https://github.com/titansec/OpenWAF>

⁴ <https://github.com/zecure/shadowd>